Online Food Ordering System

API & Micro Services

A Cornerstone Project Report submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY
in
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

K. Swarna Latha	24B11AI151
K. Sasi Meghana	24B11AI194
M. Tejo Hasini	24B11AI223
O. Bheemeswari	24B11AI305

Under the Guidance of

Mrs. Khasimbee Shaik M.Tech, (Ph.D)
Assistant Professor, AIML

ADITYA
UNIVERSITY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

2024-2028

ADITYA UNIVERSITY

School of Computing

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that the project work entitled **"Online Food Ordering System"** is submitted by

K. Swarna Latha (24B11AI151)

K. Sasi Meghana (24B11AI194)

M. Tejo Hasini (24B11AI223)

O. Bheemeswari (24B11AI305)

in partial fulfilment of the requirements for the award of the B.Tech degree in Artificial intelligence and machine learning for the academic year 2026-2027.

Project Guide

Mrs. Khasimbee Shaik, M.Tech,(Ph.D),
Assistant Professor, AIML

Head of the Department

Dr. K. Naga Bhargavi M.Tech, Ph.D.
Associate Professor & HOD, AIML

External Examiner

DECLARATION

We hereby declare that the project entitled **"Online Food Ordering System"** is original and is submitted to **ADITYA UNIVERSITY, Surampalem** in partial fulfilment of the requirements for the award of the B.Tech degree in Artificial intelligence and machine learning. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem

Date: 16/10/2026

by
K. Swarna Latha (24B11AI151)
K. Sasi Meghana (24B11AI194)
M. Tejo Hasini (24B11AI223)
O. Bheemeswari (24B11AI305)

ACKNOWLEDGEMENT

It is with immense pleasure that we would like to express our indebted gratitude to our project guide **Mrs. Khasimbee Shaik**, Department of Artificial Intelligence and Machine Learning, who has guided us a lot and encouraged us in every step of the project work. Her valuable moral support and guidance throughout the project helped us to a greater extent.

We extend our sincere gratitude to **Dr. B. Manikyala Rao (Dy.HOD)** for his continuous inspiration and for arranging the essential facilities and resources required for our project.

We are grateful to **Dr. K. Naga Bhargavi (HOD),** for inspiring us all the way and for arranging all the facilities and resources needed for our project.

We wish to thank our **Dr. M.V. Rajesh**, Associate Dean (School of Computing), **Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro ViceChancellor (Academics)** and **Dr. G. Suresh, Registrar**, for their encouragement and support during the course of our project.

We would like to extend our sincere thanks to **Dr. M. B. Srinivas, Vice- Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor and Management, Aditya University** for their unconditional support in providing us with the best infrastructural facilities and state-of-the-art laboratories during our project.

Not to forget, **Non-Teaching Staff and our Friends** who have directly or indirectly supported us in completing this project on time.

ABSTRACT

The Online Food Ordering System is a web-based, API-driven application that helps customers browse restaurants and food menus, select suitable food items, and place orders through a single platform. In traditional food-ordering systems, customers often need to contact restaurants manually to place orders and enquire about menu items, prices, and availability. This process can be time-consuming and error-prone. Customers may face difficulties in modifying orders, calculating bills, and tracking order status. The proposed system simplifies the entire process by providing a centralized platform to manage users, menu items, carts, orders, and billing through RESTful APIs.

A customer can browse food categories such as Biryani, Pizza, Burgers, and South Indian, filter items based on vegetarian or non-vegetarian preferences, ratings, and delivery time, and add selected items to the cart with cooking instructions. The system allows customers to apply coupons and view an itemized bill that includes item totals, delivery fees, platform fees, taxes, and discounts. Orders can be created, retrieved, updated, and cancelled according to their status. The application is built using Spring Boot, REST APIs, Spring Data JPA, and MySQL. Menu pagination improves the retrieval of large numbers of food items, while transaction management maintains consistency during order processing. Billing is handled through a SOAP service integrated with the order-processing workflow. AOP logging, input validation, and global exception handling improve the reliability and maintainability of the application. Additionally, a group bill-splitting module allows friends to divide the total bill equally or according to individual food items.

Keywords: Online Food Ordering, REST API, SOAP Billing, Menu Pagination, Transaction Management, Spring Boot, Spring Data JPA, MySQL, AOP Logging, Input Validation, Exception Handling, Coupon Management, Order Processing, Bill Splitting.

LIST OF CONTENTS

Chapter 1: Introduction 4
1.1 Introduction 4
1.2 Problem Statement 4
1.3 Objectives of the Project 5
1.4 Scope of the Project 6
Chapter 2: System Overview 7
2.1 Existing System 7
2.2 Limitations of Existing System 7
2.3 Proposed System 8
2.4 Software Requirements 8
2.5 Hardware Requirements 9
Chapter 3: System Implementation 10
3.1 Overview of Development Tools 10
3.2 Frontend Implementation 10
3.3 Backend Implementation 11
3.4 Database Implementation 12
3.5 Order Processing and Billing 13
3.6 Validation, Exception Handling and Logging 13
Chapter 4: System Architecture & Design 15
4.1 System Architecture 15
4.2 Database Design 16
4.3 Working Flow 16
Chapter 5: Module Description 18
5.1 User Management Module 18
5.2 Restaurant and Food Menu Module 18
5.3 Cart Management Module 18
5.4 Order Management Module 18
5.5 Billing and Coupon Module 19
5.6 Order Tracking Module 19
5.7 Bill Splitting Module 19
5.8 Validation and Error Handling Module 19
Chapter 6: API Design and Testing 20
6.1 REST API Overview 20
6.2 Menu API and Pagination 21
6.3 Order Management APIs 21
6.4 SOAP Billing Service 22
6.5 API Testing Using Postman 22
6.6 Error Handling and HTTP Status Codes 23

Chapter 7: Deployment 24
7.1 Deployment Overview 24
7.2 Application Packaging and Execution 24
7.3 Database Configuration 24
7.4 Cloud Deployment 25
Chapter 8: Output Screens 26
8.1 Home Page 26
8.2 Restaurant and Food Menu 27
8.3 Cart and Checkout Page 27
8.4 Order Confirmation and Tracking 28
8.5 Billing and Bill Splitting 29
8.6 Postman API Responses 30
Chapter 9: Conclusion 32
Chapter 10: Future Scope 33
Chapter 11: References 34

CHAPTER 1
INTRODUCTION

1.1 Introduction

Online food ordering has grown rapidly, and customers today can choose from a wide variety of food items offered by restaurants through online platforms. Although this convenience is useful, customers may face difficulties when selecting suitable food items, checking prices, and placing orders. In traditional food-ordering systems, customers often need to contact restaurants directly to enquire about menu items, availability, delivery charges, and discounts. This process can be time-consuming and may lead to incorrect orders, billing errors, and difficulties in modifying orders.

The Online Food Ordering System is developed to make food selection and ordering easier and more convenient. It is an API-driven application that manages users, restaurant menus, food items, carts, orders, and billing through REST APIs. Instead of relying on manual ordering, the system allows customers to browse food categories such as Biryani, Pizza, Burgers, and South Indian dishes, filter items based on vegetarian or non-vegetarian preferences, ratings, and delivery time, and select the food items they want. Customers can add items to their cart, provide cooking instructions, apply coupons, and review the total bill before placing an order.

The system also provides an organized order-processing mechanism. Customers can create, retrieve, update, and cancel orders according to their order status. The application is developed using Spring Boot, REST APIs, Spring Data JPA, and MySQL. Menu pagination helps retrieve food items efficiently, while transaction management maintains consistency during order processing. Billing is handled through a SOAP service integrated with the order-processing workflow. In addition, AOP logging, input validation, and global exception handling improve the reliability and maintainability of the application. A group bill-splitting module also allows friends to divide the total bill equally or according to the food items selected by each person. These features make the Online Food Ordering System a convenient and efficient solution for managing food orders.

1.2 Problem Statement

In the present digital world, online food ordering has become a convenient way for customers to purchase food from restaurants. However, traditional food-ordering methods often involve manual communication, which can lead to incorrect orders, delays in order processing, billing

mistakes, and difficulties in checking food availability. Customers may also find it difficult to compare food prices, apply discounts, calculate delivery charges, and track their orders. Managing a large number of food items and customer orders manually can increase the workload for restaurant staff and reduce the efficiency of the ordering process.

The Online Food Ordering System is proposed to overcome these problems by providing a centralized, web-based platform for browsing menus and placing food orders. The system allows customers to search and filter food items, select quantities, add items to the cart, apply coupons, and view an itemized bill before confirming an order. It uses REST APIs to manage users, menu items, and orders, while Spring Boot, Spring Data JPA, and MySQL support backend processing and data storage. Menu pagination enables efficient retrieval of food items, and transaction management helps maintain data consistency during order processing.

Furthermore, the system integrates a SOAP-based billing service to handle billing operations and uses input validation, global exception handling, and AOP logging to improve reliability. It also provides a bill-splitting feature that allows groups of customers to divide the total bill equally or according to individual food items. Therefore, the proposed system aims to make food ordering faster, more accurate, convenient, and efficient for both customers and restaurant administrators.

1.3 Objectives of the Project

- **To manage user, food menu, and order information** in a centralized MySQL database.
- **To provide REST APIs** for browsing food menus, searching food items, managing carts, placing orders, and retrieving order details.
- **To allow customers to browse and order food** through an easy-to-use web interface.
- **To manage food categories and menu items** by displaying food names, prices, and availability.
- **To provide cart and checkout facilities** that allow customers to add food items, change quantities, and review their orders before confirmation.
- **To calculate accurate bills** based on food prices, quantities, delivery charges, taxes, and discounts.
- **To manage customer orders** by allowing orders to be created, retrieved, updated, and cancelled when permitted.
- **To maintain accurate customer and order information** in the MySQL database.

- **To integrate SOAP-based billing services** for generating bill information and supporting order processing.
- **To improve application reliability** through input validation, exception handling, and logging.
- **To retrieve food menu items efficiently** using pagination.
- **To provide a bill-splitting facility** that allows customers to divide the total bill equally or according to individual food items.

1.4 Scope of the Project

The scope of the Online Food Ordering System includes the development of a complete web-based platform that allows customers to browse food menus, select food items, and place orders conveniently. The system covers user management, food categories, menu browsing with search and filtering, cart management, coupon application, bill calculation, order placement, and order management. It also provides supporting features such as menu pagination, order tracking, SOAP-based billing, and group bill splitting, allowing customers to divide the bill equally or according to individual food items.

The project uses Spring Boot to develop the backend application and REST APIs, Spring Data JPA to perform database operations, and MySQL to store user, menu, and order information. SOAP services are integrated with the billing process. Transaction management helps maintain data consistency during order processing, while input validation, exception handling, and AOP logging improve the reliability and maintainability of the application. The user interface allows customers to browse food items, add items to the cart, review bills, and manage their orders.

Features such as online payment gateway integration, live GPS-based delivery tracking, advanced AI-based food recommendations, loyalty and reward programs, and integration with external restaurant and delivery platforms are outside the current scope of the project. These features can be added in future versions of the system.

CHAPTER 2
SYSTEM OVERVIEW

2.1 Existing System

In the existing system, customers generally order food by visiting restaurants, checking printed menus, or using basic online food-ordering platforms. They select food items from the available menu, check the prices, and place their orders. In some cases, customers need to contact the restaurant to confirm food availability, delivery charges, and order details. Billing is often handled separately, and customers may not always receive a clear breakdown of the total amount before placing an order.

Restaurant staff manage food menus, prices, availability, and customer orders. When these activities are handled manually, mistakes may occur while recording food items, quantities, and customer details. Customers may also face difficulties in modifying orders, applying discounts, checking order status, and calculating the final bill. Managing a large number of orders can become time-consuming for restaurant staff.

The existing approach is therefore time-consuming and may lead to incorrect orders, billing errors, and delays in order processing. Customers may find it difficult to compare food items, understand additional charges, and divide bills when ordering with friends. These limitations highlight the need for a centralized Online Food Ordering System that simplifies menu browsing, order placement, billing, and order management through a single platform.

2.2 Limitations of Existing System

- **No centralized ordering:** Customers may need to contact restaurants to check menus, prices, and food availability.
- **Manual order errors**: Incorrect food items or quantities may be recorded while placing orders.
- **Limited menu information:** Customers may find it difficult to check food prices, categories, and availability in one place.
- **Difficult order modification:** Changing or cancelling an order may require contacting restaurant staff.
- **Billing problems:** Customers may not receive a clear breakdown of food costs, delivery charges, taxes, and discounts.

- **No efficient order tracking:** Customers may have difficulty checking the current status of their orders.
- **Time-consuming process:** Manual order management increases the workload of restaurant staff and may cause delays.
- **Difficult bill splitting**: Dividing the total bill among friends may require manual calculations.
- **Limited data management**: Managing large numbers of food items and customer orders manually can lead to inconsistent records.

2.3 Proposed System

The proposed system is the Online Food Ordering System, a Spring Boot-based application that helps customers browse food items, select suitable meals, and place orders easily through a single platform. Customers can explore food categories such as Biryani, Pizza, Burgers, and South Indian dishes, filter items based on their preferences, add food items to the cart, apply coupons, and view the final bill before placing an order.

The backend is built with Spring Boot and exposes REST APIs for menu management, customers, orders, and billing operations. Spring Data JPA is used for database operations with MySQL. Every input is validated using Bean Validation, errors are handled through a global exception handler that returns clear messages, and AOP logging records service operations. Transaction management helps maintain consistent order data, while menu pagination supports efficient access to large numbers of food items.

The system overcomes the limitations of the existing approach by providing itemized billing that includes food costs, delivery charges, platform fees, taxes, and discounts. Customers can create, view, update, and cancel orders according to the system rules. A SOAP billing service is integrated with order processing, and a bill-splitting feature allows friends to divide the total amount equally or according to their selected food items. These features make the ordering process more convenient, organized, and reliable than traditional manual ordering methods.

2.4 Software Requirements

- **Operating System:** Windows or Linux
- **Programming Languages:** Java 17 and JavaScript
- **Backend Framework:** Spring Boot 3
- **API Style:** REST API with JSON

- **Database:** MySQL 8
- **ORM:** Spring Data JPA with Hibernate
- **Frontend:** HTML, CSS, and JavaScript
- **Build Tool:** Maven
- **IDE:** IntelliJ IDEA or Visual Studio Code
- **API Testing Tool:** Postman
- **Version Control:** Git and GitHub

2.5 Hardware Requirements

- **Processor:** Intel Core i3 or higher
- **RAM:** Minimum 4 GB
- **Storage:** Minimum 20 GB free disk space
- **System Type:** 64-bit operating system
- **Peripherals:** Keyboard, mouse, and monitor
- **Internet Connection:** Required for downloading dependencies and for cloud deployment

CHAPTER 3
SYSTEM IMPLEMENTATION

3.1 Overview of Development Tools

3.1.1 Development Environment

The Online Food Ordering System was developed using IntelliJ IDEA for the Java and Spring Boot backend, as it provides code completion, debugging, and strong Maven integration. Visual Studio Code was used to write and test the HTML, CSS, and JavaScript files of the user interface. MySQL Workbench was used to create the database and to view the stored tables and records during development.

Postman was used to send requests to the REST API endpoints and to examine the responses. Maven was used to manage dependencies and to build the project into an executable file, while Git and GitHub were used for version control and for sharing the source code among the team members.

3.1.2 Frameworks and Libraries Used

The main framework used in the Online Food Ordering System is Spring Boot, which simplifies the development of stand-alone Java applications with embedded Tomcat, automatic configuration, and starter dependencies. The project uses spring-boot-starter-web for REST controllers, spring-boot-starter-data-jpa for database access, spring-boot-starter-validation for Bean Validation, and spring-boot-starter-aop for Aspect-Oriented Programming. The MySQL Connector/J driver enables communication between the application and the MySQL database.

Spring Data JPA, built on Hibernate, allows database tables to be represented as Java entity classes and reduces the need to write SQL queries manually. Repository interfaces provide ready-made methods such as save(), findAll(), and findById(), along with custom query methods based on method names. These features are used to manage customer details, food menu items, orders, and billing information. Jackson, included with Spring Boot, converts Java objects into JSON responses and JSON requests into Java objects, enabling communication between the frontend and the REST APIs.

3.2 Frontend Implementation

The frontend of the Online Food Ordering System is developed using HTML, CSS, and JavaScript and is placed in the static resources folder of the Spring Boot project, so that it is

served by the same application that provides the APIs. The interface is designed to make food ordering simple and convenient. The home page displays food categories and available food items, while the navigation bar allows customers to move between different sections of the application.

The Menu page displays food items such as Biryani, Pizza, Burgers, and South Indian dishes, along with their prices and descriptions. Customers can search for food items and filter them based on categories and preferences. The Cart page allows customers to add or remove items, change quantities, and provide special cooking instructions. The Coupon section allows customers to apply available discount codes. The Billing page displays an itemized bill containing food costs, delivery charges, platform fees, taxes, and discounts. The Order Management page allows customers to place orders, view order details, and update or cancel orders according to the system rules.

The interface also provides a bill-splitting feature that helps customers divide the total amount equally or according to the selected food items. The frontend communicates with the backend through REST APIs to retrieve menu details, submit orders, and display billing information. Clear messages are displayed whenever the backend rejects a request, such as when invalid input is provided or an order cannot be processed. The pages use responsive layouts so that they can be viewed on both computers and mobile phones.

3.3 Backend Implementation

The backend is developed using Java and Spring Boot and follows a layered structure. The controller layer receives HTTP requests through REST controllers, validates the input using the @Valid annotation, and passes the requests to the service layer. The service layer contains the business logic of the Online Food Ordering System, including food menu management, cart operations, order processing, billing calculations, coupon validation, and bill splitting. The repository layer, implemented using Spring Data JPA interfaces, performs database operations for storing and retrieving customer details, food items, and order information from MySQL.

The REST APIs provide the main operations required by the application, including retrieving food menu items, searching and filtering food items, managing cart items, placing orders, retrieving order details, updating or cancelling orders, and calculating bills. Data transfer objects are used to define the information required in API requests and responses. These objects help transfer customer details, food item information, order data, and billing details between the frontend and backend.

When the application starts for the first time, sample food items and related data can be inserted into an empty database to support testing and demonstration. The menu contains different food categories, such as Biryani, Pizza, Burgers, and South Indian dishes, along with their prices and descriptions. The backend also integrates a SOAP billing service with order processing. Transaction management helps maintain consistent order records, while Bean Validation, global exception handling, and AOP logging improve the reliability and maintainability of the application.

3.4 Database Implementation

The system uses a MySQL database accessed through Spring Data JPA. The tables are created or updated automatically from the entity classes when the application starts, because the property spring.jpa.hibernate.ddl-auto is set to update. The database must be configured before running the application, which makes it easier to manage and maintain the project.

The database contains five main tables. The users table stores customer details such as name, email, and phone number. The menu_item table stores each food item with its name, category, price, vegetarian status, rating, delivery time, and availability. The orders table stores order information, including the user, the applied coupon, the order status, special instructions, the bill amount, and the creation time. The order_item table stores the food items, quantities, and prices included in each order, and the coupon table stores coupon codes, discounts, minimum order amounts, and availability. These tables help organize customer information, food menu details, and order records.

Table	Main Columns	Purpose
users	id, name, email, phone	Stores customer details
menu_item	id, name, category, price, veg, rating, delivery_time, available	Stores food items and availability
orders	id, user_id, coupon_id, status, special_instructions, bill_amount, created_at	Stores orders, billing totals, and order status
order_item	id, order_id, menu_item_id, quantity, price	Stores the items in each order
coupon	id, code, discount, min_order, active	Stores coupon codes and eligibility rules

3.5 Order Processing and Billing

The order-processing workflow is an important part of the Online Food Ordering System and is implemented in the service layer. It helps customers select suitable food items, manage their cart, calculate the bill, and place orders. The workflow runs in the following stages.

- **Stage 1 –** Food Selection: Customers browse the available food items, such as Biryani, Pizza, Burgers, and South Indian dishes. They select food items based on their preferences, prices, and availability.
- **Stage 2 –** Cart Management: The selected food items are added to the cart. Customers can change the quantity, remove items, and provide special cooking instructions before proceeding.
- **Stage 3 –** Price Calculation: The system calculates the total cost by multiplying each item's price by its quantity and adding the costs of all selected items. Delivery charges, platform fees, and applicable taxes are also included.
- **Stage 4 –** Coupon Application: Customers can apply available coupon codes to receive discounts. The system validates the coupon and updates the bill according to the applicable discount rules.
- **Stage 5 –** Order Confirmation: The system displays the final bill and allows customers to confirm their order. After successful validation, the order details are stored in the database, and the order status is updated for further processing.
- **Stage 6 –** Feedback and Tracking: Customers can view their order details and track the order status. The system displays relevant information to help customers understand the progress of their food order.

Stages 3 to 5 are carried out inside a single transactional service method, so an order is either saved completely or not saved at all if any check fails. The final bill is calculated by the SOAP billing service, which is described in Chapter 6.

3.6 Validation, Exception Handling and Logging

Input validation is performed using Bean Validation annotations in the Online Food Ordering System. The food item name must not be blank, the quantity must be valid, the customer details must be provided, the email must be in a valid format, and the food item identifier must be specified when required. If a request violates any of these rules, the system returns an HTTP 400 (Bad Request) response with a message identifying the invalid field.

Exception handling is centralized in a class annotated with @RestControllerAdvice. A custom exception can carry an HTTP status and a message, allowing the service layer to report problems such as a food item that does not exist (404 Not Found), invalid order details (400 Bad Request), and an attempt to access an unavailable order (404 Not Found). A conflict, such as an invalid order-state transition, can return HTTP 409 (Conflict), depending on the implemented business rules. The exception handler converts these errors into clear JSON responses that the frontend can display to customers.

Aspect-Oriented Programming (AOP) is used for logging. An @Around advice can be applied to service-layer methods to record the method name and the time taken to execute it in milliseconds. This keeps logging code separate from the main business logic and provides useful information for monitoring, debugging, and maintaining the application.

CHAPTER 4
SYSTEM ARCHITECTURE & DESIGN

4.1 System Architecture

The Online Food Ordering System follows a layered three-tier architecture consisting of a presentation layer, an application layer, and a database layer. The presentation layer is the web interface through which customers browse food items, manage their cart, apply coupons, place orders, and view billing details. The application layer is the Spring Boot backend, which is divided into controller, service, and repository layers. The database layer is the MySQL database that stores customer details, food items, orders, order items, and coupons.

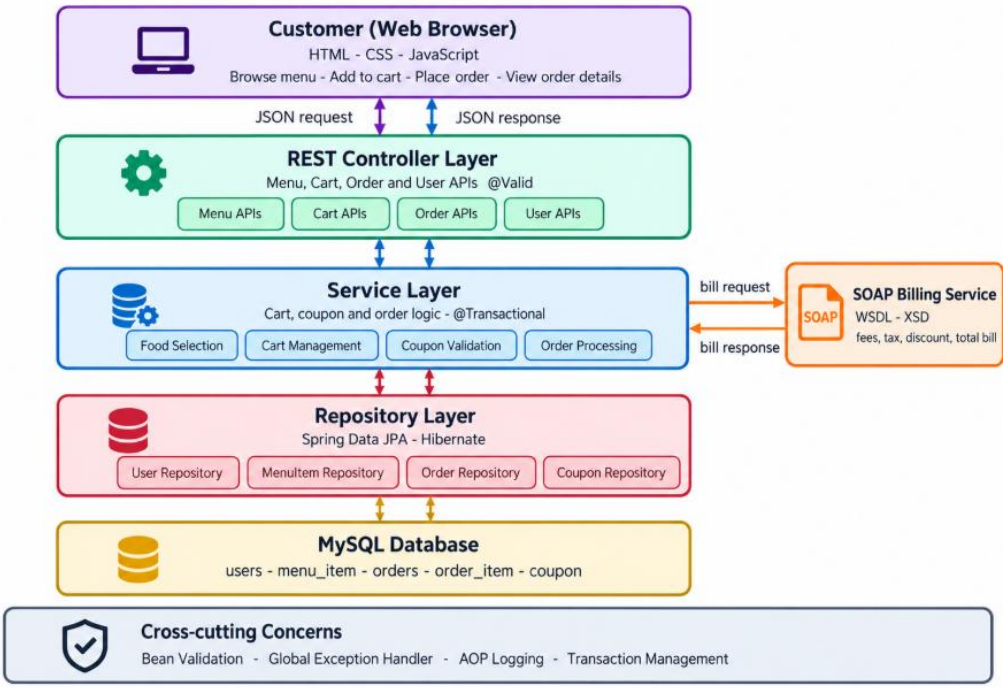


Figure 4.1: System Architecture of the Online Food Ordering System

The web interface sends JSON requests to the REST controller. The controller validates the request and calls the service layer, where food selection, cart management, coupon validation, billing calculations, and order processing are carried out. The service layer uses the repository layer to read and write data in MySQL. The result is returned to the interface as a JSON response and displayed as menu details, cart information, billing details, order status, or a confirmation message. The SOAP billing service can be integrated with the service layer to calculate the final bill, including applicable fees, taxes, and discounts. Validation, exception handling, transaction management, and AOP logging support the application across these layers.

4.2 Database Design

The database design of the Online Food Ordering System is kept simple and organized to reduce duplication of data. The users table stores customer details such as user ID, name, email, and phone number. The orders table stores order information, including the user ID, coupon ID, order status, special instructions, bill amount, and order creation date. The coupon table stores coupon codes, discount details, minimum order amount, and coupon availability. These tables are connected through primary keys and foreign keys to maintain relationships between customers, orders, and coupons.

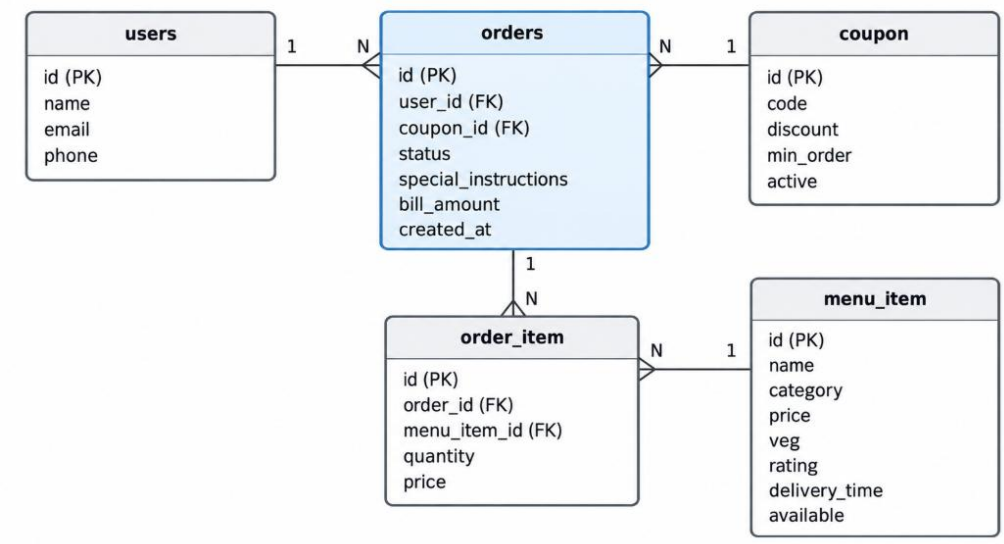


Figure 4.2: Entity Relationship Diagram

The order_item table stores the individual food items included in each order, along with the order ID, menu item ID, quantity, and price. The menu_item table contains food details such as item name, category, price, vegetarian or non-vegetarian status, rating, delivery time, and availability. One user can place multiple orders, while each order can contain multiple order items. Each order item refers to a specific menu item, and a coupon can be associated with multiple orders according to the system's rules. This database structure helps manage customer information, food menus, orders, discounts, and billing efficiently. New food items and coupons can be added by inserting records into the relevant tables without changing the overall database structure.

4.3 Working Flow

The working flow of the Online Food Ordering System begins when the customer opens the application and views the available food menu. The customer selects food items, chooses the

required quantities, and adds them to the cart. The system checks the availability of the selected items and calculates the total cost. The customer can apply a coupon, provide special cooking instructions, and review the final bill before confirming the order. After successful validation, the order is saved in the MySQL database, and an order confirmation is displayed.

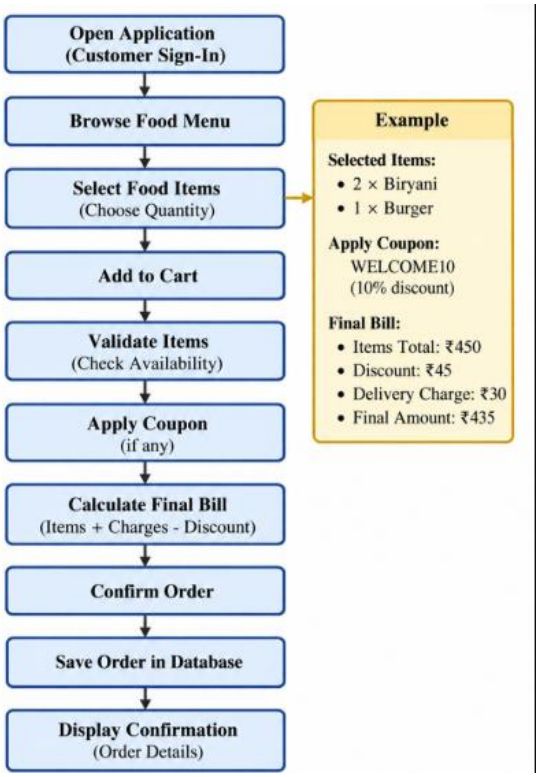


Figure 4.3: Working Flow of the System

For example, consider a customer who selects two Biryanis and one Burger. The system calculates the cost based on the prices and quantities of the selected items. If the customer enters a valid coupon code, the applicable discount is deducted from the bill. The system then validates the order details and checks food availability. If all checks are successful, the order is stored in the database with its total amount and status. If an item is unavailable or the input is invalid, the system displays an appropriate error message and asks the customer to correct the request.

CHAPTER 5
MODULE DESCRIPTION

5.1 User Management Module

The User Sign-In Module collects the name and email address of the customer. These details help identify the customer while placing food orders. The email format is validated, and customer information is stored in the database. After signing in, the customer's name can be displayed in the navigation bar, making the ordering experience more personalized.

5.2 Restaurant and Food Menu Module

The Food Menu Module is the main module of the system. It allows customers to browse available food items, such as Biryani, Pizza, Burgers, and South Indian dishes. Customers can search for food items, filter them by category, check prices, and view availability. Each food item displays details such as its name, price, category, description, and other available information.

The Search and Filter Module helps customers find suitable food items quickly. Customers can search by food name and filter items based on categories, vegetarian or non-vegetarian preferences, ratings, and delivery time, where supported. This module reduces the time required to find a preferred food item from the available menu.

This module also organizes customer details and food menu information. Customer records include details such as name, email, and phone number, while menu records contain food names, categories, prices, descriptions, and availability. The system uses MySQL and Spring Data JPA to store, retrieve, and manage these records efficiently.

5.3 Cart Management Module

The Cart Management Module allows customers to add selected food items to their cart before placing an order. Customers can increase or decrease quantities, remove unwanted items, and provide special cooking instructions. The system updates the item totals based on the selected quantities and prepares the cart for checkout.

5.4 Order Management Module

The Order Management Module handles the complete order-processing workflow. It receives the customer's selected food items, quantities, and special instructions. Before saving the order, the system checks the availability of the selected items and validates the required information.

Once the order is successfully placed, the order details are stored in the database and a confirmation is displayed to the customer.

5.5 Billing and Coupon Module

The Billing and Coupon Module calculates the total amount payable for an order. It considers food item prices, quantities, delivery charges, platform fees, applicable taxes, and discounts. Customers can enter a coupon code, and the system checks whether the coupon is valid before applying the discount. The final bill is displayed before order confirmation. A SOAP-based billing service can be integrated to support billing operations.

5.6 Order Tracking Module

The Order Tracking Module allows customers to view their order details and check the current order status. The system displays the available status information so that customers can understand the progress of their orders. This module improves transparency and helps customers manage their food orders conveniently.

5.7 Bill Splitting Module

The Bill Splitting Module helps groups of customers divide the total food bill. The total amount can be divided equally among the members or according to the food items selected by each person. This reduces manual calculations and makes it easier for friends or groups to share the cost of a food order.

5.8 Validation and Error Handling Module

The Validation and Error Handling Module checks customer inputs and order details before processing requests. It validates required fields, quantities, customer information, food item availability, and coupon details. The system uses centralized exception handling to return clear error messages when a request cannot be processed. AOP logging records service operations and execution times, helping developers monitor and debug the application. These features improve the reliability and maintainability of the Online Food Ordering System.

CHAPTER 6
API DESIGN AND TESTING

6.1 REST API Overview

The Online Food Ordering System uses REST APIs to enable communication between the web interface and the Spring Boot backend. The interface sends HTTP requests whenever a customer loads the food menu, searches for food items, adds items to the cart, applies a coupon, places an order, or views order details. The backend processes each request, interacts with the MySQL database, and returns the response in JSON format.

The APIs follow standard HTTP conventions: GET is used for retrieving data, POST is used for creating data or placing orders, PUT or PATCH is used for updating existing data, and DELETE is used for deleting data when supported. Appropriate HTTP status codes, such as 200 (OK), 201 (Created), 400 (Bad Request), 404 (Not Found), and 409 (Conflict), are returned so that the interface can respond correctly to successful operations and errors. These APIs help manage food menus, customers, carts, orders, coupons, and billing operations efficiently.

API Endpoints

Method	Endpoint	Purpose	Success
GET	/api/menu-items	List all available food items	200
GET	/api/menu-items?category=	Retrieve food items by category	200
GET	/api/menu-items?q=	Search food items by name	200
GET	/api/coupons	List all active coupons	200
POST	/api/coupons/apply	Validate and apply a coupon	200
GET	/api/cart	View items in the cart	200
POST	/api/cart/items	Add a food item to the cart	201
DELETE	/api/cart/items/{id}	Remove a food item from the cart	200
POST	/api/orders	Place a new food order	201
GET	/api/orders/{id}	Retrieve order details using the order ID	200
PUT	/api/orders/{id}	Update order details, if permitted	200
DELETE	/api/orders/{id}	Cancel an order, if permitted	200
POST	/api/billing/calculate	Calculate the final bill for selected items	200

6.2 Menu API and Pagination

A restaurant menu can contain a large number of food items, so returning all of them in a single response would be slow. The menu API therefore supports pagination. The endpoint GET /api/menu-items accepts the optional parameters page, size, and sort, together with the category and q filters described above. The repository uses Spring Data pagination and returns one page of food items at a time.

The response contains the food items of the requested page along with the page number, the page size, the total number of items, and the total number of pages. This lets the interface load the menu page by page and keeps the response small and fast, even when the menu grows. A sample request is shown below.

GET /api/menu-items?category=Biryani&page=0&size=5&sort=price

6.3 Order Management APIs

The Order Processing API receives the customer ID, the list of selected food items, their quantities, and any special instructions or coupon code. A sample request is shown below.

Sample Request:

{ "userId": 101, "items": [{ "menuItemId": 1, "quantity": 2 }, { "menuItemId": 3, "quantity": 1 }], "couponCode": "FOOD10", "specialInstructions": "Less spicy" }

For this request, the system processes the selected food items and calculates the bill based on their prices and quantities. If the coupon is valid, the applicable discount is applied to the order. The response contains the order ID, selected food items, total bill amount, discount details, and order status. After successful processing, the order details are stored in the MySQL database, and an order confirmation is returned to the customer.

The order creation API receives the customer ID, selected food items, their quantities, and any special instructions or coupon code. It first checks whether the customer and selected food items exist, then verifies the availability of the food items and validates the coupon, if provided. If all required checks pass, the order is saved in the database and the system returns HTTP status 201 (Created). A sample request is shown below.

Sample Request:

{ "userId": 101, "items": [{ "menuItemId": 4, "quantity": 2 }], "couponCode": "FOOD10", "specialInstructions": "Less spicy" }

For this request, the system creates an order for two quantities of the selected food item, applies the coupon if it is valid, calculates the final bill, and stores the order details in the MySQL database. The response includes the order ID, total bill amount, and order status.

6.4 SOAP Billing Service

Billing is provided by a SOAP web service that is integrated with order processing. The service is described by a WSDL file and an XSD schema. When an order is placed, the service layer sends the item total, the delivery charge, and the coupon code to the billing service, which returns the bill breakdown. A sample request is shown below.

```
<CalculateBillRequest>
    <itemsTotal>450</itemsTotal>
    <deliveryCharge>30</deliveryCharge>
    <couponCode>FOOD10</couponCode>
</CalculateBillRequest>
```

For the example in Figure 4.3, the items total 450, the 10% coupon gives a discount of 45, and the delivery charge is 30, so the final amount is 435. Platform fees and taxes are added according to the configured rates. If the coupon is invalid or the request is incorrect, the service returns a SOAP fault, and the order service rolls back the transaction so that no incomplete order is stored.

6.5 API Testing Using Postman

The REST APIs of the Online Food Ordering System are tested using Postman to verify that requests and responses work correctly. The test cases cover both valid and invalid inputs to confirm the successful operation of each endpoint and the proper handling of errors. The main test cases are listed below.

Test Case	Input	Expected Result
Load food menu	GET /api/menu-items	200 with available food items
Search food items	GET /api/menu-items?q=biryani	200 with matching food items
View coupons	GET /api/coupons	200 with active coupons
Add item to cart	POST /api/cart/items (valid item and quantity)	201 Created
Place valid order	Valid customer ID, food items and quantities	201 Created with order details
Invalid quantity	Quantity = 0	400 with validation message

Test Case	Input	Expected Result
Unavailable food item	Order an unavailable food item	400 with error message
Invalid coupon	Enter an invalid coupon code	400 with error message
Order not found	Request an unknown order ID	404 with message
Invalid customer	Submit an unknown customer ID	404 with message

Postman testing helps verify communication between the frontend APIs and the backend, including database operations and error handling. It confirms that the system returns appropriate responses for valid requests and meaningful error messages for invalid requests.

6.6 Error Handling and HTTP Status Codes

When a request cannot be processed, the global exception handler returns the matching HTTP status code and a clear JSON message that the interface can display directly. The main error conditions are listed below.

Condition	Status	Message
Invalid field value	400	Field name with reason
Food item unavailable	400	Food item is currently unavailable
Invalid coupon	400	Invalid or expired coupon
Customer not found	404	Customer not found
Food item not found	404	Food item not found
Order not found	404	Order not found
Duplicate order request	409	Order request conflicts with the current state

CHAPTER 7
DEPLOYMENT

7.1 Deployment Overview

The Online Food Ordering System is packaged as a single executable application. Since the web interface is placed inside the Spring Boot project, the same application serves both the frontend pages and the REST APIs. Only the Spring Boot application and the MySQL database need to be configured for deployment. This approach simplifies the deployment process and reduces the chances of configuration errors between separate frontend and backend applications. Once the application is started successfully, customers can access the system through a web browser to browse food items, add items to the cart, place orders, and view billing details.

7.2 Application Packaging and Execution

The Online Food Ordering System is built using Maven, which manages project dependencies and builds the application. The command mvn clean package compiles the source code, runs the configured tests, and generates an executable JAR file. The application can then be started using the java -jar command, which launches the embedded Tomcat server on port 8080. During development, the command mvn spring-boot:run can be used to start the application directly without manually creating the JAR file. After the application starts successfully, users can open the server address in a web browser to access the system. The application allows customers to browse food items, manage their cart, place orders, and view billing details through the web interface.

7.3 Database Configuration

The database connection for the Online Food Ordering System is configured in the application.properties file, which contains the MySQL database URL, username, and password. The database URL specifies the database to be used by the application. The property spring.jpa.hibernate.ddl-auto is set to update, allowing Hibernate to create or update database tables based on the entity classes when the application starts. The system stores information about customers, food items, and orders in the MySQL database. Sample food items, such as Biryani, Pizza, Burgers, and South Indian dishes, can be inserted automatically during the initial setup if data initialization is configured. This configuration helps maintain the database structure and supports efficient storage and retrieval of food-ordering information.

7.4 Cloud Deployment

For online access, the Online Food Ordering System can be deployed on any cloud platform that supports Java and Spring Boot, such as Render or Railway. The MySQL database can be hosted using a managed database service such as Aiven. The database URL, username, and password should be configured as environment variables in the cloud platform instead of being directly written into the source code, helping protect sensitive information. After successful deployment, customers can access the Online Food Ordering System through its public URL without installing or running the application on their own computers. They can browse food items, add items to the cart, place orders, and view order details online.

CHAPTER 8
OUTPUT SCREENS

This chapter presents the output screens of the Online Food Ordering System. These screens demonstrate the complete food-ordering workflow, starting from the home page and food menu browsing, followed by food selection, cart management, checkout, order confirmation, billing, and order tracking. The screens also demonstrate features such as coupon application and bill splitting, showing how the system helps customers place and manage their food orders conveniently.

8.1 Home Page

The home page of the Online Food Ordering System introduces customers to the BiteBliss platform. It features a modern, responsive navigation bar, a promotional hero banner highlighting fast delivery and hygienic food, quick cuisine category selectors (Biryani, Pizza, Burgers, South Indian, Desserts, and Beverages), a live food search bar, delivery location controls, and top restaurant listings.

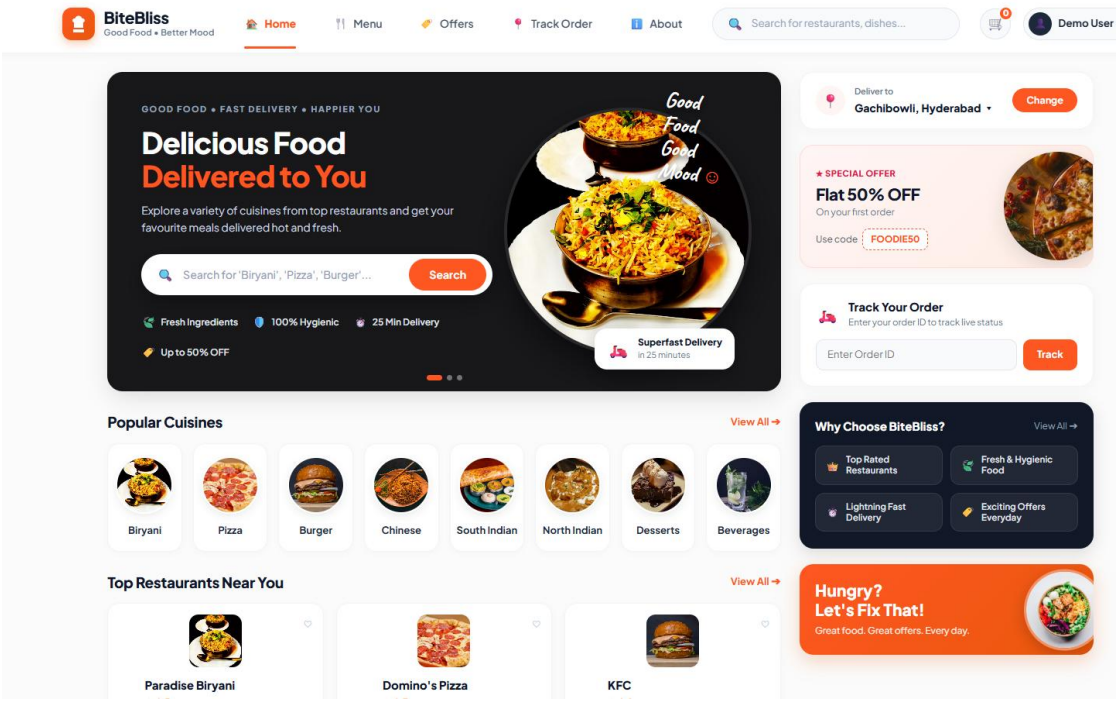


Figure 8.1: Home Page of Online Food Ordering System (BiteBliss)

8.2 Restaurant and Food Menu

The restaurant and food menu screen displays the complete food catalog. Customers can search for specific dishes by name or ingredients, filter items by category (such as Biryani & Rice, Pizzas, Burgers & Wraps, North Indian, and South Indian), and apply preference filters including Pure Veg, Non-Veg, Top Rated (4.5+), and Under 20 Minutes delivery. Each dish card presents the item name, restaurant name, pricing, discount tags, rating, and an Add to Cart button.

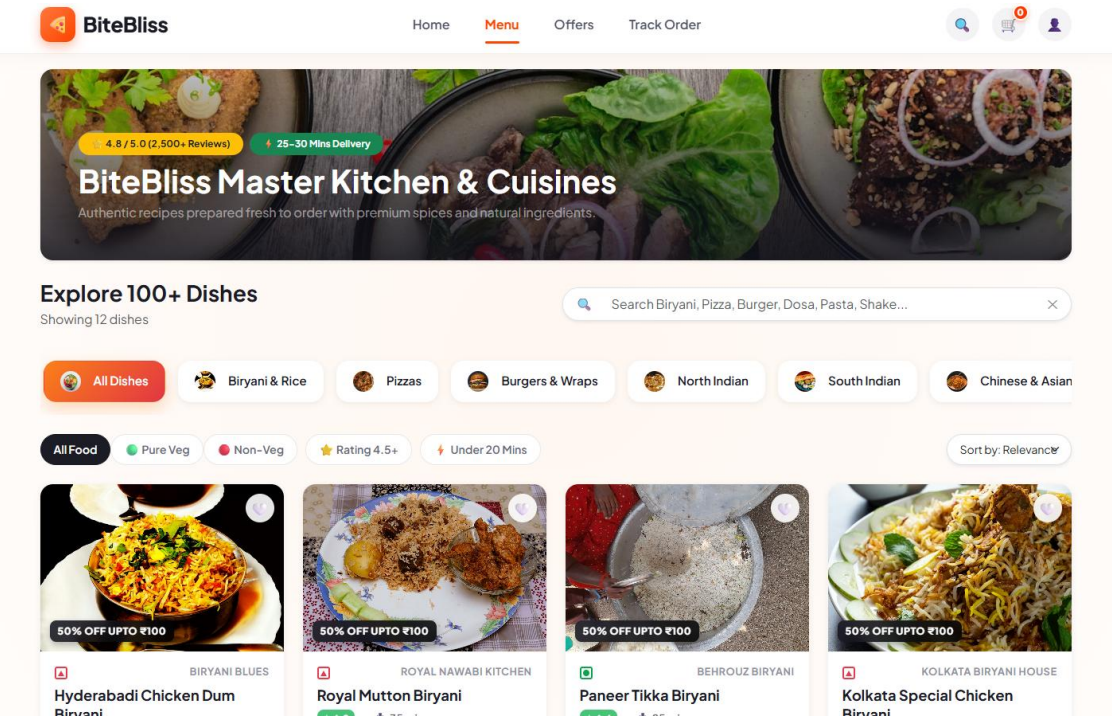


Figure 8.2: Restaurant and Food Menu Page with Category Filters and Dishes

8.3 Cart and Checkout Page

The cart and checkout workflow enables customers to review their selected food items, adjust item quantities, remove items, add special cooking instructions (such as 'Less spicy' or 'Extra green chutney'), and apply promotional coupons. Once the cart is confirmed, the customer proceeds to checkout to select their saved delivery address (Home or Work) and choose a secure payment method, including Instant UPI/QR code, Credit/Debit card, and Net Banking.

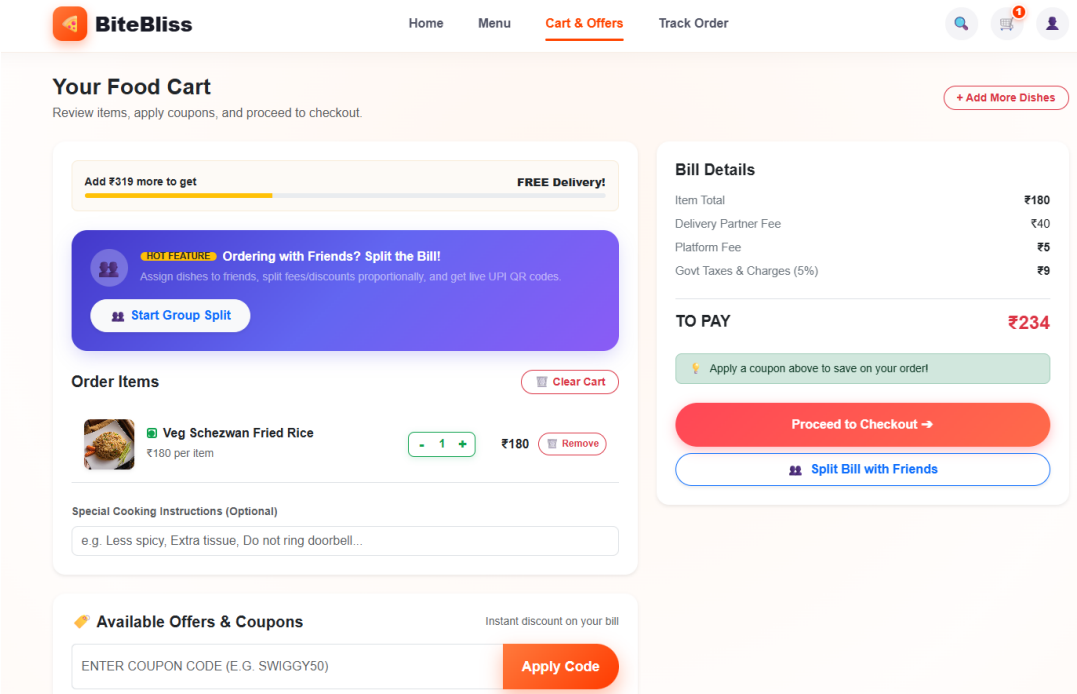


Figure 8.3 (a): Cart Management with Cooking Instructions and Applied Coupon

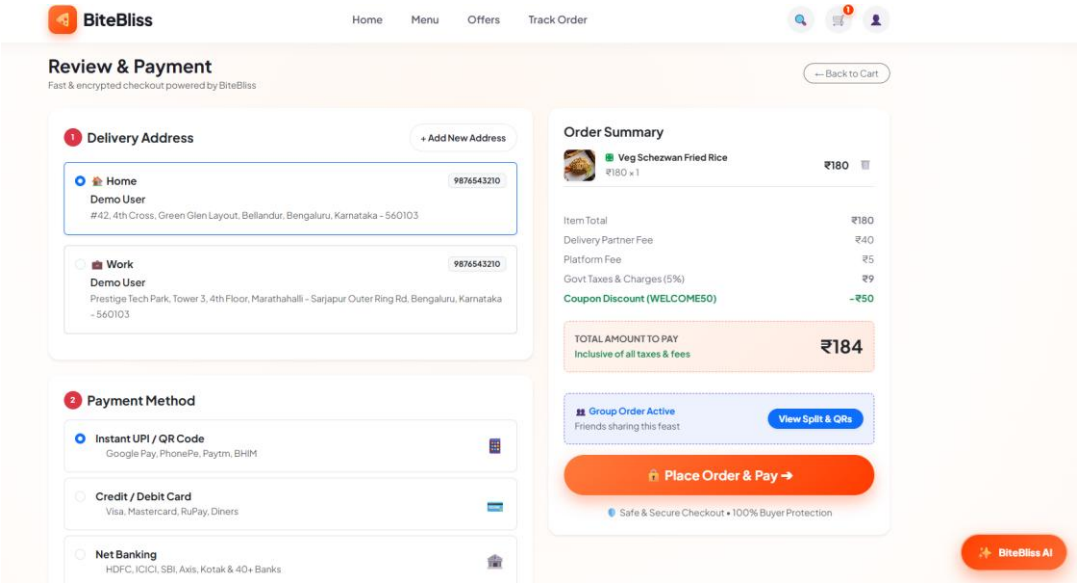


Figure 8.3 (b): Delivery Address and Payment Method Selection at Checkout

8.4 Order Confirmation and Tracking

Upon successful order placement, the system displays the order confirmation screen with the generated Order ID and real-time order status (such as PENDING). A multi-stage interactive tracking timeline visualizes the order progression from Order Placed, Order Accepted, Preparing, Ready, Out for Delivery, to Delivered. Live polling actively monitors backend status transitions and displays status history updates.

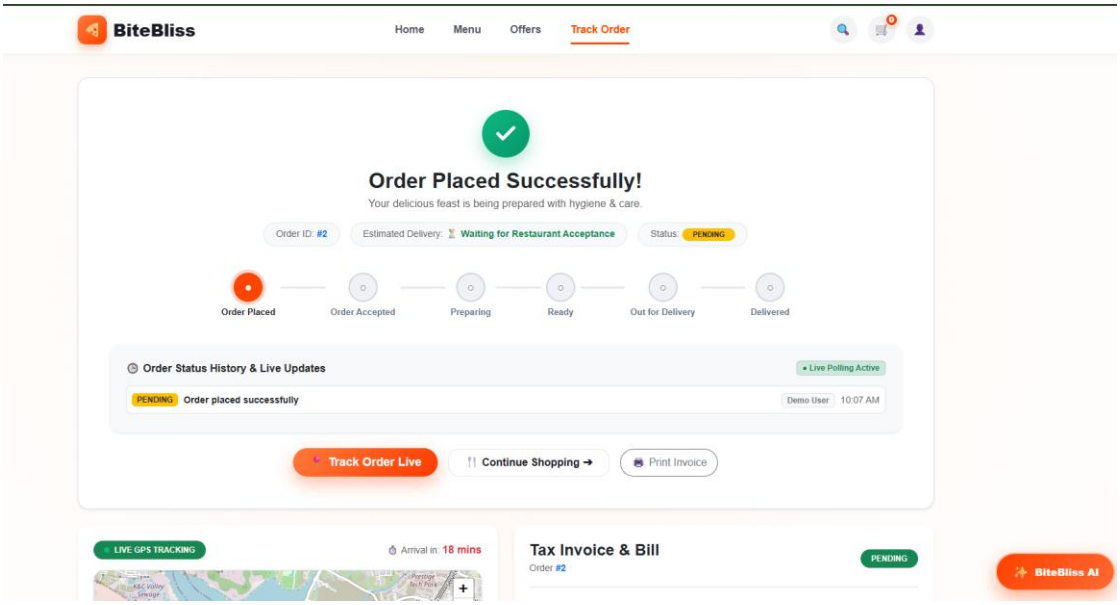


Figure 8.4: Order Confirmation and Live Order Status Tracking

8.5 Billing and Bill Splitting

The billing and bill-splitting module provides a complete tax invoice detailing the item subtotal, delivery partner fee, platform fee, government taxes (GST 5%), coupon discount deductions, and the final amount paid. In addition, the integrated group bill-splitting feature divides the total bill among group members (e.g., dividing ₹184 into equal shares of ₹46 per person) and generates dynamic UPI QR payment buttons for each member, while displaying live GPS delivery partner tracking.

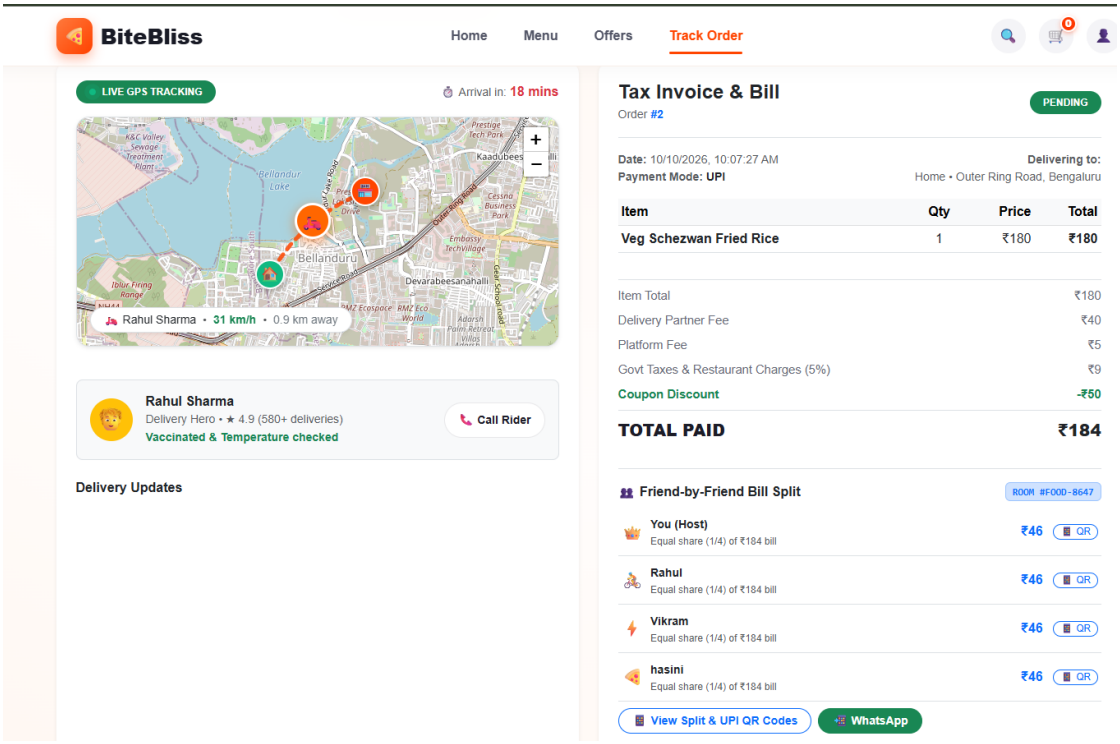


Figure 8.5: Itemized Tax Invoice, Billing Breakdown, and Group Bill Splitting

8.6 Postman API Responses

The REST API endpoints of the Online Food Ordering System were thoroughly tested using Postman. The test suite validates endpoint communication, request payload validation, and JSON response formatting. The screen below illustrates a successful POST request to the /api/orders endpoint, returning HTTP status 201 Created with the serialized OrderResponse JSON payload confirming user details, itemized totals, discount calculations, and order creation.

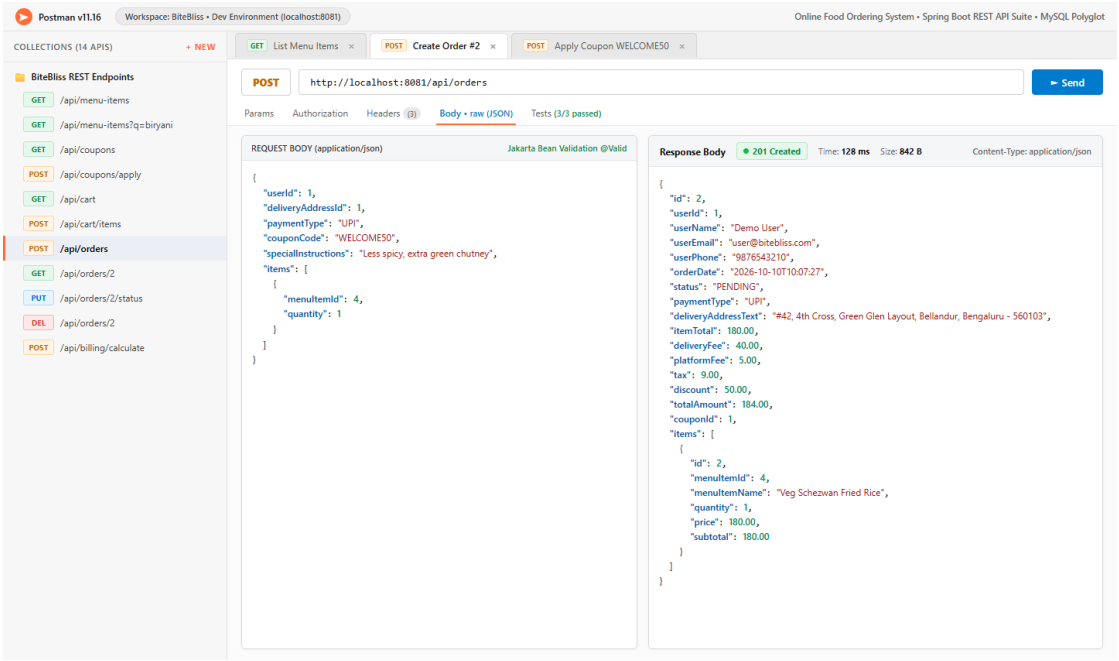


Figure 8.6: Postman REST API Testing and JSON Responses (Order Creation Endpoint)

CHAPTER 9
CONCLUSION

The Online Food Ordering System was developed to make the food ordering process easier, faster, and more convenient for customers. The system reduces the difficulties associated with traditional food ordering by providing a centralized platform where customers can browse food menus, check prices, select food items, manage their carts, and place orders. It helps reduce manual errors, simplifies billing, and provides better management of customer orders.

The project provides REST APIs for food menu management, customer details, order processing, and billing operations. Features such as menu search and pagination, coupon application, itemized billing, order management, SOAP billing integration, validation, centralized exception handling, and AOP logging improve the functionality and reliability of the application. The bill-splitting feature also helps customers divide the total amount among friends when placing group orders.

The project successfully demonstrates the integration of Spring Boot, REST APIs, Spring Data JPA, and a MySQL database using a layered architecture. It provides a practical solution for managing online food orders in an organized manner. The system can be further enhanced with online payment integration, live delivery tracking, AI-based food recommendations, and customer loyalty programs to improve the overall ordering experience.

CHAPTER 10
FUTURE SCOPE

- **AI/ML-Based Food Recommendations**: Use machine learning techniques to suggest food items based on customers' preferences, previous orders, ratings, and food choices.
- **Online Payment Integration:** Integrate secure payment gateways such as UPI, debit cards, credit cards, and digital wallets to support convenient online payments.
- **Live GPS-Based Delivery Tracking:** Add GPS tracking to help customers monitor the delivery partner's location and estimate the arrival time of their orders.
- **Restaurant Management Dashboard:** Provide a dashboard for restaurant administrators to manage food menus, update prices, check food availability, and manage incoming orders.
- **Order Notifications:** Send real-time notifications through email, SMS, or push notifications when an order is confirmed, prepared, dispatched, or delivered.
- **Customer Ratings and Reviews:** Allow customers to rate food items and restaurants and provide feedback based on their ordering experience.
- **Microservices Architecture:** Separate menu management, order processing, customer management, and billing into independent services to improve scalability and maintainability.
- **Advanced Bill Splitting:** Enhance the bill-splitting feature to allow customers to divide bills by selected food items, quantities, or equal shares among group members.
- **Loyalty and Reward Programs**: Introduce reward points, discounts, and special offers for regular customers to encourage customer engagement.
- **Enhanced Security:** Implement secure authentication, role-based access control, and data protection mechanisms to protect customer information and order details.

CHAPTER 11
REFERENCES

[1] Spring Boot Documentation. Available: https://docs.spring.io/spring-boot/

[2] Spring Data JPA Documentation. Available: https://docs.spring.io/spring-data/jpa/reference/

[3] Spring Framework Aspect-Oriented Programming. Available: https://docs.spring.io/spring-framework/reference/core/aop.html

[4] Jakarta Bean Validation. Available: https://beanvalidation.org/

[5] MySQL Documentation. Available: https://dev.mysql.com/doc/

[6] R. T. Fielding, Architectural Styles and the Design of Network-based Software Architectures, Doctoral Dissertation, University of California, Irvine, 2000. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

[7] Apache Maven Documentation. Available: https://maven.apache.org/guides/

[8] Postman Learning Center. Available: https://learning.postman.com/

[9] GitHub Documentation. Available: https://docs.github.com/

[10] HTML Living Standard, WHATWG. Available: https://html.spec.whatwg.org/

[11] MDN Web Docs, CSS: Cascading Style Sheets. Available: https://developer.mozilla.org/en-US/docs/Web/CSS

[12] MDN Web Docs, JavaScript Guide. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

[13] SOAP Version 1.2, W3C Recommendation. Available: https://www.w3.org/TR/soap12/

[14] Jackson Documentation. Available: https://github.com/FasterXML/jackson

[15] Git Documentation. Available: https://git-scm.com/doc